TradingAlgorithms_HW4_Documentation

برای زیاد شدن داک عذرخواهی میکنم. سعی کردم تمام موارد رو پوشش بدم و جاهایی رو هم کمتر توضیح دادم ولی به خاطر عکس ها و تکه کدها صفحات زیاد شد وگرنه خود مطالب اینقدر جا نمیگرفتند. باز پوزش میخوام.

برای تکه کدهای طولانی "Read the full in notebook ..." گذاشتم تا داک کمتر شه و کد کامل در notebook قابل مشاهده هست.

نکاتی قبل از شروع داک

سه نکته قبل از شروع کردن داک خود پروژه:

- دکتر انتظاری یک سری کد سر کلاس زده بودند و گفتند که ما هم میتونیم از این ها استفاده کنیم، من این کدهارو هم در نوت بوک مشخص کردم.
- دکتر انتظاری سر کلاس گفتند که برای بخش ششم *ema* رو روی *close* باید بزنیم و اشتباه در تمرین اومده بوده که روی high/low بزنیم.
- پنجاه کوین برتر همگی در یاهو فایننس و بایننس نبودند و نمیشد که دانلود کرد. من یه اپشن استفاده از کوین هاب بعد پنجاه تا برتر هم گذاشتم ولی ازش استفاده نکردم تا پیش فرض سوال عوض نشه. ولی خب گفتم که به این موضوع حواسم بوده و یه اپشنی گذاشتم.

بخش صفر

در این بخش قبل از خود استراتژی ها باید بریم و دیتاهای برترین ۵۰ کوین با بیشترین مارکت کپ رو بدست بیاریم.

> یک نکته‌ای که اینجا هست موارد اخر این لیست (۴۰ تا ۵۰) ثابت نیستند و هرروز ممکنه عوض بشن

لیست موقع که من دراوردم این موارد بودند:

```
COINS = [
    ("BTC","Bitcoin"), ("ETH","Ethereum"), ("USDT","Tether"),
("BNB","BNB"),
    ("XRP","XRP"), ("USDC","USDC"), ("SOL","Solana"), ("TRX","TRON"),
    ("DOGE","Dogecoin"), ("ADA","Cardano"), ("BCH","Bitcoin Cash"),
("HYPE","Hyperliquid"),
    ("LINK","Chainlink"), ("LEO","UNUS SED LEO"), ("XMR","Monero"),
("XLM","Stellar"),
    ("ZEC","Zeash"), ("USDe","Ethena USDe"), ("LTC","Litecoin"),
("SUI","Sui"),
    ("DAI","Dai"), ("AVAX","Avalanche"), ("HBAR","Hedera"), ("SHIB","Shiba
Inu"),
    ("MNT","Mantle"), ("PYUSD","PayPal USD"), ("TON","Toncoin"),
("WLFI","World Liberty Financial"),
    ("CRO","Cronos"), ("UNI","Uniswap"), ("DOT","Polkadot"),
("TAO","Bitensor"),
    ("AAVE","Aave"), ("USD1","World Liberty Financial USD"),
("CC","Canton"), ("BGB","Bitget Token"),
```

```
    ("OKB","OKB"), ("M","MemeCore"), ("ASTER","Aster"), ("NEAR","NEAR
Protocol"),
    ("ETC","Ethereum Classic"), ("PEPE","Pepe"), ("ENA","Ethena"),
("ICP","Internet Computer"),
    ("PI","Pi"), ("XAUt","Tether Gold"), ("PAXG","PAX Gold"),
("USDG","Global Dollar"),
    ("KCS","KuCoin Token"), ("WLD","Worldcoin"),
]
```

یک مشکل دیگر که اینجا خوردم همه این کوین هارو *yfinance* و نه *binance* نداشتند.
از بین این کوین ها موارد زیر یا کلا نبودند یا در تاریخ ما کامل نبودند. با TA صحبت کردم گفتند که می‌توانیم این هارو استفاده نکنیم.

```
   - USDe
   - WLFI
   - USD1
   - ASTER
   - PI
   - HYPE (Hyperliquid)
   - MNT (Mantle)
   - CC (Canton)
```

و دیتاها در فولدر **Data/** ذخیره شده‌اند.

کد دانلود کردن کوین‌ها:

```
import time
import requests
import yfinance as yf
from datetime import datetime, timezone


OUT_DIR = os.path.join(os.getcwd(), DATA_SAVE_DIR)
os.makedirs(OUT_DIR, exist_ok=True)

COINS = [
   Read the full in notebook...
]

# top 50 above are not all downladed from yahoo finance or binance so i
added some more coins after top 50 to cover it
EXTRA_COINS = [
Read the full in notebook...
]

if USE_MORE_COINS:
    ALL_COINS = COINS + [c for c in EXTRA_COINS if c[0] not in [x[0] for x
in COINS]]
```

```
else:
    ALL_COINS = COINS


Read the full in notebook...
```

برای دانلود داده‌های قیمتی کوین‌ها از دو منبع Yahoo Finance و Binance استفاده کردیم:

- ۱. `yahoo_ticker_for(symbol)` : نماد کوین را به فرمت تیکر Yahoo Finance (`SYMBOL-USD`) تبدیل می‌کند
- ۲. `download_from_yahoo(symbol)` : داده‌های OHLCV را از Yahoo Finance دریافت می‌کند. در صورت عدم موفقیت، تیکر `SYMBOL-USDT` را هم امتحان می‌کنیم
- ۳. `download_from_binance(symbol)` : برای دریافت داده‌های کندل استیک روزانه از Binance API استفاده می‌کند. به عنوان منبع دوم در صورت نداشتن کوین در Yahoo Finance
- ۴. `save_df_csv(symbol, df)` : دیتافریم را در فایل CSV با فرمت `SYMBOL_1D.csv` ذخیره می‌کند
- ۵. `download_coins()` : فانکشن اصلی که برای هر کوین ابتدا Yahoo Finance را امتحان می‌کند و در صورت وجود نداشتن یا داده ناکافی، از Binance استفاده می‌کند

بخش اول

برای این بخش نیاز داریم که روی کوین‌های بالا **ADF** تست اجرا کنیم و رمزارزای مانا رو پیدا کنیم.

```
def run_adf_on_data(data_dir='Data', alpha=0.05, start_day=None,
end_day=None):
    files = sorted(glob.glob(os.path.join(data_dir, '*_1D.csv')))
    results = []
    for f in files:

Read the full in notebook...
```

در این فانکشن هم همینکارو انجام میدیم و دانه دانه کوین هارو لود میکنیم و **ADF** تست رو روشون اجرا میکنیم.
و
دراخر نتایج رو برمیگردونیم.

```
def plot_candlestick(df, title):
    dates = df.index.to_pydatetime()
    if len(dates) == 0:
        return

    fig, ax = plt.subplots(figsize=(10, 4))
    dnum = mdates.date2num(dates)


Read the full in notebook...
```

در این فانکشن هم یک پیاده‌سازی برای رسم نمودار Candlestick داریم. داده‌های OHLC را از دیتافریم می‌گیریم و کندل‌ها را برای بازه‌های زمانی مشخص شده رسم می‌کنیم و رنگ‌بندی سبز برای صعودی و قرمز برای نزولی روشن قرار میدیم.

نتایج این پارت به صورت زیر می‌باشد:

```
Stable coins (p-value) in the period from 2024-12-01 to 2025-12-01:
PYUSD: 0.000000
DAI: 0.000000
DOT: 0.000150
ZEC: 0.000327
USDT: 0.000996
USDC: 0.007032
WLD: 0.015210
NEAR: 0.016447
TAO: 0.019186
XLM: 0.027161
ICP: 0.028728
BGB: 0.029622
SHIB: 0.030948
LTC: 0.047629
XRP: 0.048759
```

بخش دوم

```
def compute_metrics_for_coin(filepath):
    name = os.path.basename(filepath).replace('_1D.csv', '')

    df = pd.read_csv(filepath, index_col=0, parse_dates=True)

Read the full in notebook...
```

در این فانکشن برای هر کوین، فایل مربوطه را لود می‌کنیم و پس از فیلتر کردن بازه زمانی (Start/End Day)، متریک‌های **Half-Life** و **Hurst Exponent** را برای آن سری زمانی محاسبه می‌کنیم و خروجی را به همراه دیتافریم فیلتر شده برمی‌گردانیم.
دو فانکشن half_life و hurst_exponent هم جز کدهای سر کلاس می‌باشند.

در این بخش یک سوال شده بود: ۵. توضیح دهید توان Hurst و نیمه‌عمر چه چیزی درباره سری‌های قیمتی بیان می‌کنند.

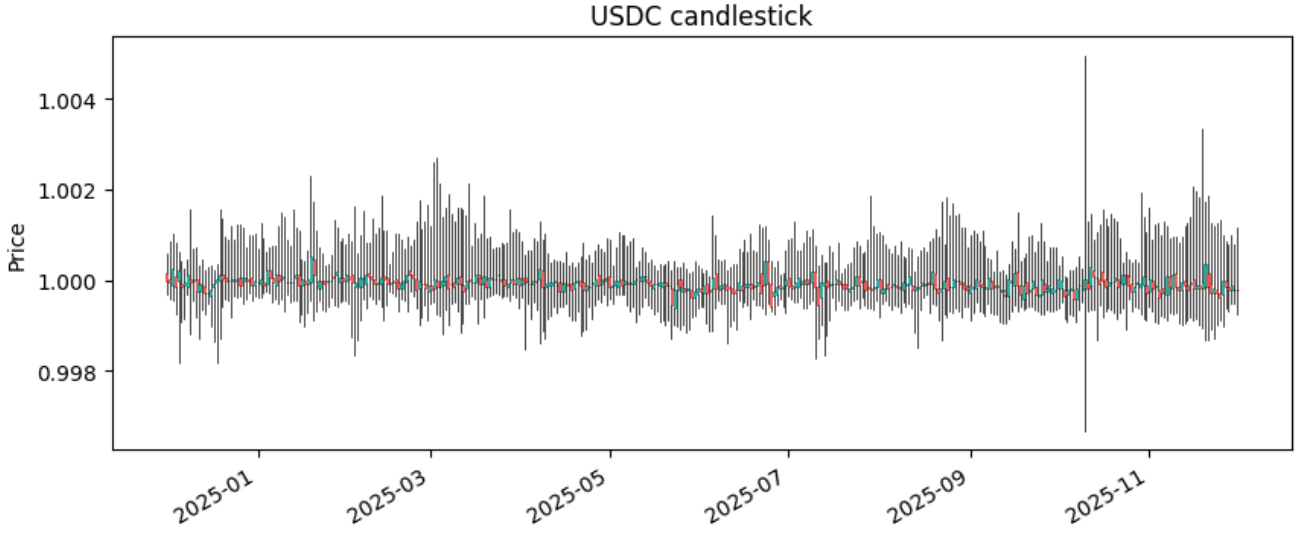
جواب

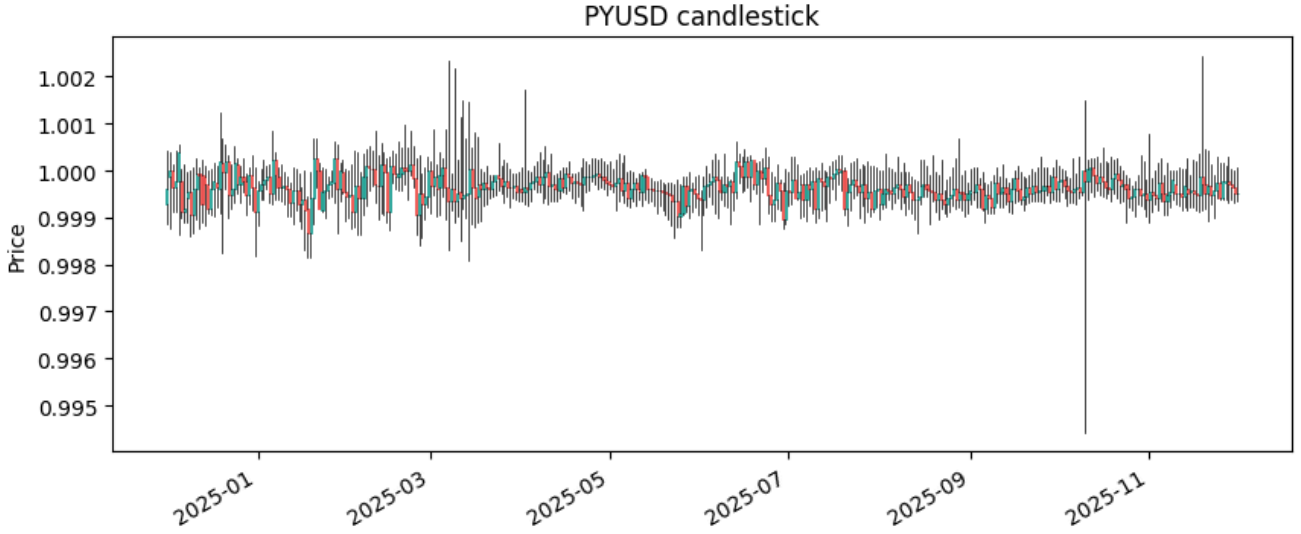
- **توان هرست (H):** این معیار به ما می‌گوید که رفتار یک سری زمانی (مانند اسپرد قیمت) چگونه است:
- **H<0.5: بازگشت به میانگین** سری (Mean-Reverting) است. یعنی اگر قیمت بالا برود، احتمالا دوباره پایین می‌آید (مناسب برای جفت‌ارزها)
- **H=0.5: گام تصادفی** سری (Random Walk) است. حرکت بعدی قابل پیش‌بینی نیست

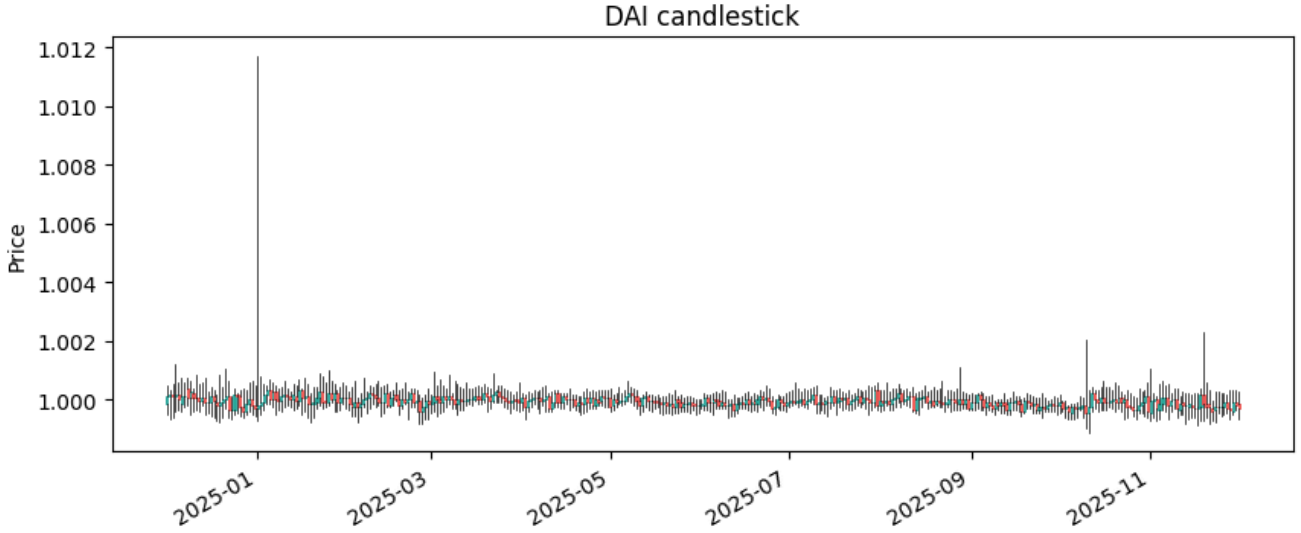
است. یعنی اگر قیمت بالا برود، تمایل دارد به بالا رفتن ادامه دهد (Trending) سری **روندار :H>0.5**

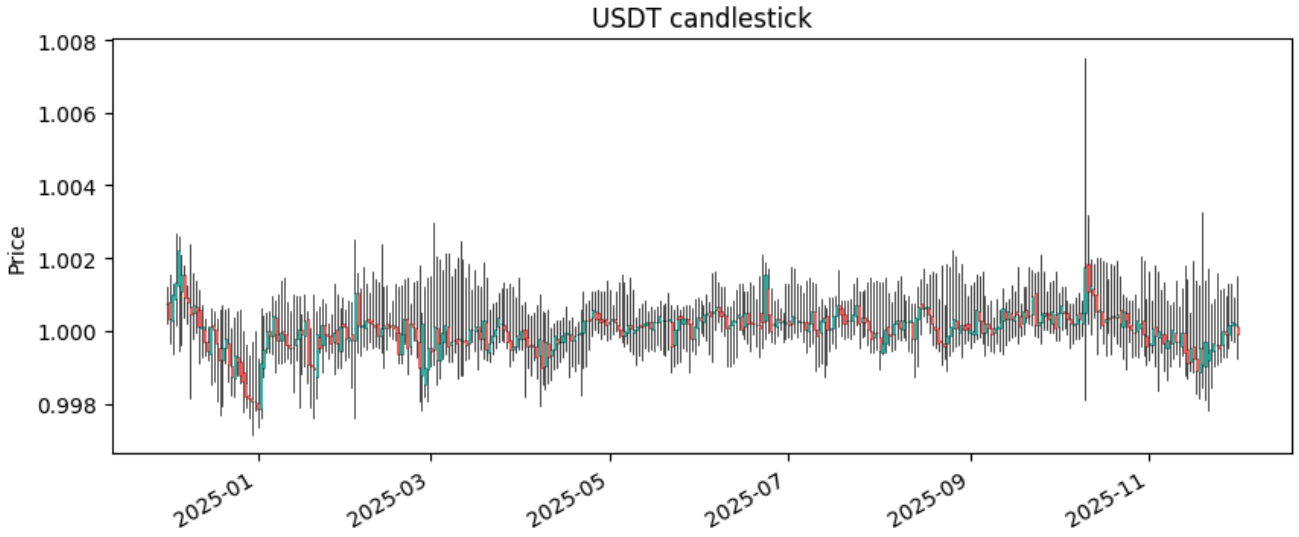
- **نیمه‌عمر (Half-Life):** این معیار نشان می‌دهد چقدر طول می‌کشد تا سری پس از دور شدن از میانگین، نیمی از مسیر بازگشت را طی کند.
  - **نیمه‌عمر کوتاه:** اسپرد سریع به میانگین برمی‌گردد (فرصت‌های معاملاتی بیشتر، ریسک کمتر).
  - **نیمه‌عمر طولانی:** زمان زیادی طول می‌کشد تا قیمت برگردد (فرصت کمتر، ریسک خواب سرمایه).

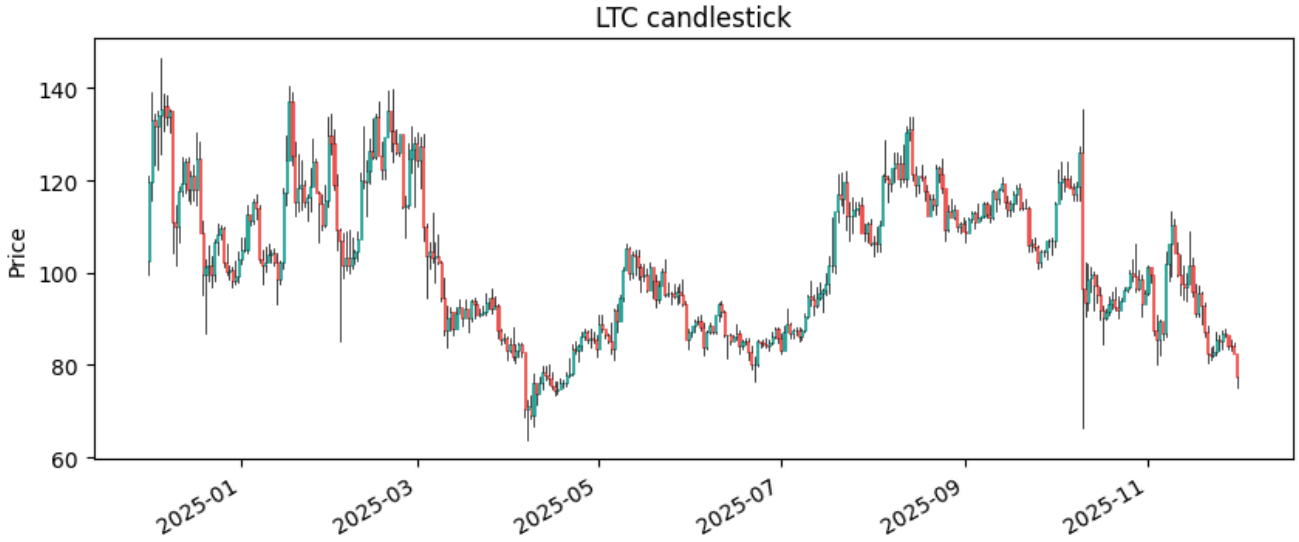
حال اگر ران کنیم، نتایج به صورت زیر هستند:

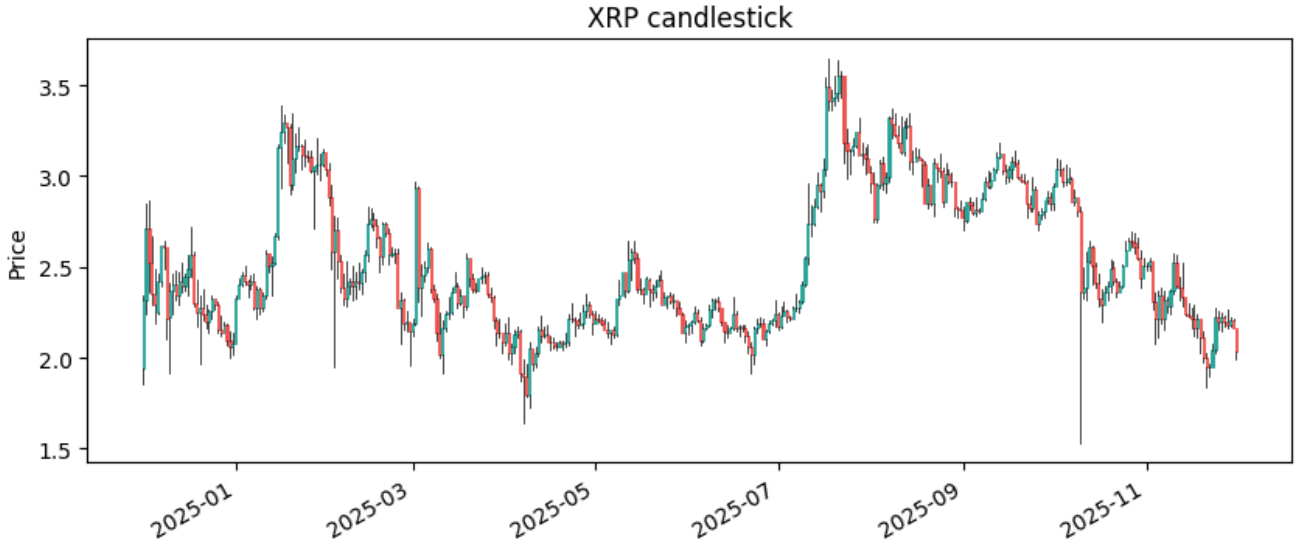


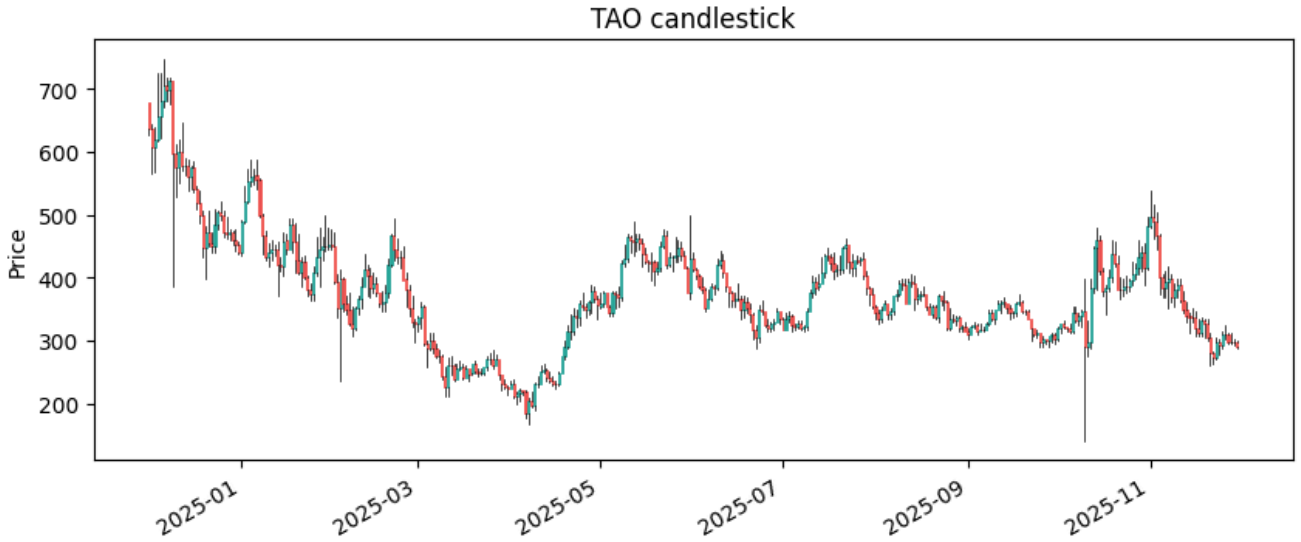





USDT candlestick
Price
1.008
1.006
1.004
1.002
1.000
0.998
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11


LTC candlestick
Price
140
120
100
80
60
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11


XRP candlestick
Price
3.5
3.0
2.5
2.0
1.5
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11



.
.
.

بخش سوم

قبل از پیدا کردن سری‌های قیمتی نامانا نیاز به چند *pre processing* نیاز داریم.

ابتدا با اجرای تست ADF، کوین‌هایی که مانا هستند را جدا می‌کند و سپس یک دیکشنری می‌سازیم تا بتونیم به سریع به کوین مورد نیاز دسترسی داشته باشیم.(از فایل لود نکنیم.)

```
def prepare_candidates(data_dir, alpha, start_day, end_day):
    results, stationary_names = run_adf_on_data(data_dir=data_dir,
alpha=alpha, start_day=start_day, end_day=end_day)
    nonstationary = [name for name, p, n in results if name not in
stationary_names]
    files_map = {os.path.basename(p).replace('_1D.csv', ''): p for p in
sorted(glob.glob(os.path.join(data_dir, '*_1D.csv')))}
    return nonstationary, files_map
```

حال، این فانکشن دیتاهای مربوط به یک Combo را لود می‌کند. نکته مهم اینجا، Align کردن زمانی داده‌هاست. یعنی Intersection ایندکس‌های زمانی تمام کوین‌های آن ترکیب را می‌گیرد تا مطمئن شویم قیمت‌ها دقیقا مربوط به یک روز هستند.

در ادامه نیاز داریم که روی کلوزهای تراز شده، تست جوهانسن را اجرا کنیم:

```
def load_and_align_combo(combo, files_map, start_day, end_day):
    dfs = []

    for name in combo:
        path = files_map.get(name)
        if path is None:
            return None
```

```
Read the full in notebook...
```

این فانکشن در واقع یک Wrapper برای تست **Johansen** است که خروجی آن مشخص می‌کند آیا ترکیب Cointegrated هست یا خیر. همچنین برای ۵ تست اول، نتایج را جهت دیباگ چاپ می‌کند.

حالا باید متریک‌های آماری اسپرد را محاسبه کنیم. برای تمیزی کد، بخش OLS و محاسبات متریک‌ها را جدا کردیم:

```
def check_cointegration(closes, combo, tested_count):
    res, is_coint, vec, trace0, cv95 = johansen_test(closes)

    cv95_v1 = cv95
    cv95_v2 = np.nan

    if tested_count ≤ 5:
        print(f"  Combo {combo}: trace0={trace0:.2f}, cv95[0,1]={cv95_v1
if not np.isnan(cv95_v1) else float('nan'):.2f}, cv95[0,2]={cv95_v2 if not
np.isnan(cv95_v2) else float('nan'):.2f}, pass={is_coint}")

    return is_coint, vec, trace0, cv95
```

در اینجا:

- سری زمانی اسپرد را با ضرب ماتریسی وزن‌ها می‌سازیم.
- تست ADF را روی خود اسپرد اجرا می‌کنیم تا از مانایی آن مطمئن شویم.
- متریک‌های Half-Life و Hurst Exponent را محاسبه می‌کنیم.

```
def calculate_spread_metrics(closes, vec, idx):
    spread = closes.dot(vec)

    adf_p = perform_adf_test(spread)

    spread_series = pd.Series(spread, index=idx)

    hl = half_life(spread_series)
    try:
        H = hurst_exponent(spread_series.values)
    except Exception:
        H = np.nan

    return spread, {'adf_p': adf_p, 'hl': hl, 'hurst': H}
```

حال می‌توانیم *ohlc* ساختگی را درست کنیم.
فقط یک نکته در اینجا هست:

> محاسبه `High` و `Low` در اینجا strait forward نیست. اگر وزن یک کوین منفی باشد، تاثیر آن برعکس می‌شود. یعنی `Low` آن کوین باعث افزایش `High` اسپرد می‌شود و بالعکس.

- ماتریس ستونی از قیمت‌ها ساخته می‌شود: `opens` , `highs` , `lows` , `closes` (شکل T×n).
- `Open = opens.dot(vec)` و `Close = closes.dot(vec)` .
- برای `High` و `Low` ، اگر وزن مؤلفه مثبت باشد اثر مستقیم (high→high, low→low) اضافه می‌شود. اگر وزن منفی باشد علامت وارونه شده و برای محاسبه‌ی `High` از `low` آن مؤلفه و برای `Low` از `high` آن مؤلفه استفاده می‌شود (برای حفظ معنی بیشینه/کمینه ترکیبی).

که کد این فانکشن هم به صورت زیر است:

```
def build_synthetic_ohlc(dfs, idx, vec, closes_mat):
    opens = np.column_stack([d.loc[idx, 'Open'].values for d in dfs])
    highs = np.column_stack([d.loc[idx, 'High'].values for d in dfs])
    lows = np.column_stack([d.loc[idx, 'Low'].values for d in dfs])

    spread_ohlc = pd.DataFrame(index=idx)
    spread_ohlc['Open'] = opens.dot(vec)
    spread_ohlc['Close'] = closes_mat.dot(vec)

Read the full in notebook...
```

حال تنها نیاز به یک فانکشن انتهایی داریم که تمام فانکشن های بالا را بهم وصل کند.
منطق آن را هم میتونیم به صورت زیر خلاصه کنیم:

- لیست نامانا‌ها را بگیریم.
- کوین‌های استیبل را حذف کنیم.
- ترکیبات مختلف را بسازیم و تک‌تک تست کنیم.
- اگر ترکیبی cointegrated بود، متریک‌ها و OHLC آن را محاسبه و ذخیره کنیم.

یک اپشن شافل کردن هم اضافه کردم چون فضای سرچ به شدت بزرگه و منطقا نمیتونیم کل ان را جست و جو کنیم یک ترکیب رندوم از آن را سرچ کنیم.(در بخش ۶ خیلی تاثیر گذار بود)

```
def print_final_summary(tested, found):
    print(f'\nTested {tested} combos, found {len(found)} cointegrated.')
    print('\nAll found combos and their half-life values:')
    for i, f in enumerate(found):
        print(f"  {i+1}. {f['names']}: hl={f['hl']}, adf_p=
{f['adf_p']:.6f}")

def find_linear_stationary_combinations(data_dir=DATA_SAVE_DIR,
alpha=0.05, target=10, max_size=10, start_day=None, end_day=None ,
shuffle=False):
    nonstationary, files_map = prepare_candidates(data_dir, alpha,
start_day, end_day)
```

```
Read the full in notebook...
```

> دراینجا چند استیبل کوین رو برداشتم. وقتی اینارو برنمیداشتم بعضی اسپردهایی پیدا میشد که درست نبودند و جواب‌هایی بدی میگرفتم. منطقی بود که اینارو حذف کنیم و استفاده نکنیم.

```
def select_top5_and_plot(found):
    if not found:
        print("no cointegrated combos found.")
        return []



Read the full in notebook...
```

در این فانکشن نهایی، هدف ما گلچین کردن بهترین کاندیداها از بین تمام ترکیبات پیدا شده و تصویرسازی آن‌هاست.

منطق کار به این صورت است که:

۱. ابتدا مواردی که `Half-Life` آن‌ها به هر دلیلی محاسبه نشده (`Nan` یا `None`) را فیلتر می‌کنیم.

۲. لیست باقی‌مانده را بر اساس **Half-Life** مرتب می‌کنیم.

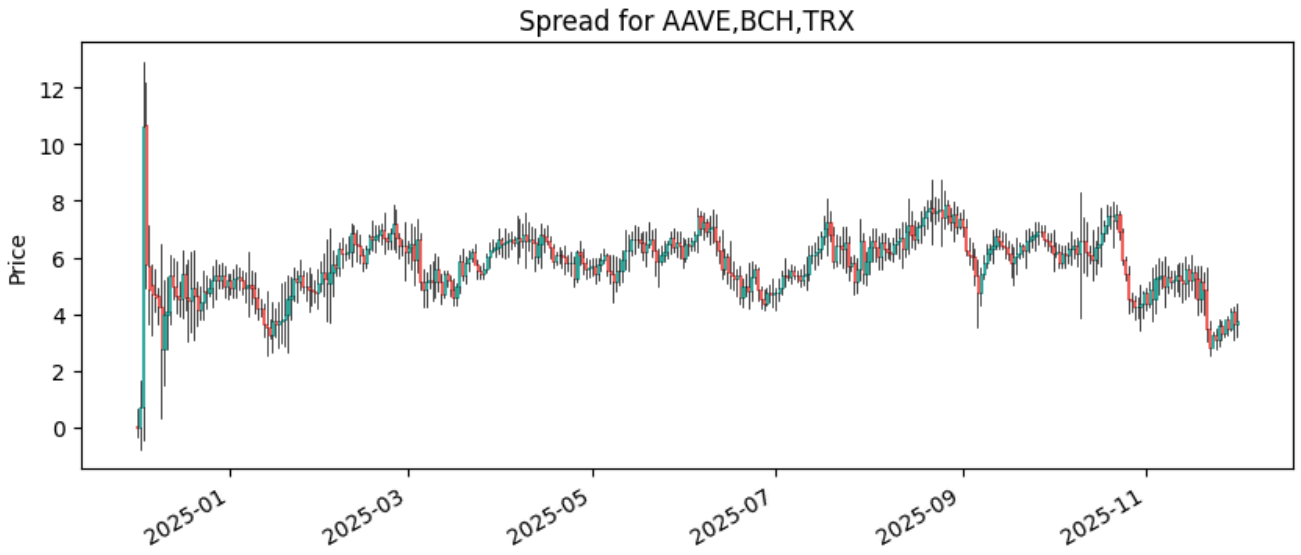
۳. برای هر کدام از این ۵ مورد، دو نوع نمودار رسم می‌کنیم: - **نمودار اول (Constituents):** قیمت تک‌تک کوین‌های موجود در آن ترکیب را رسم می‌کنیم.

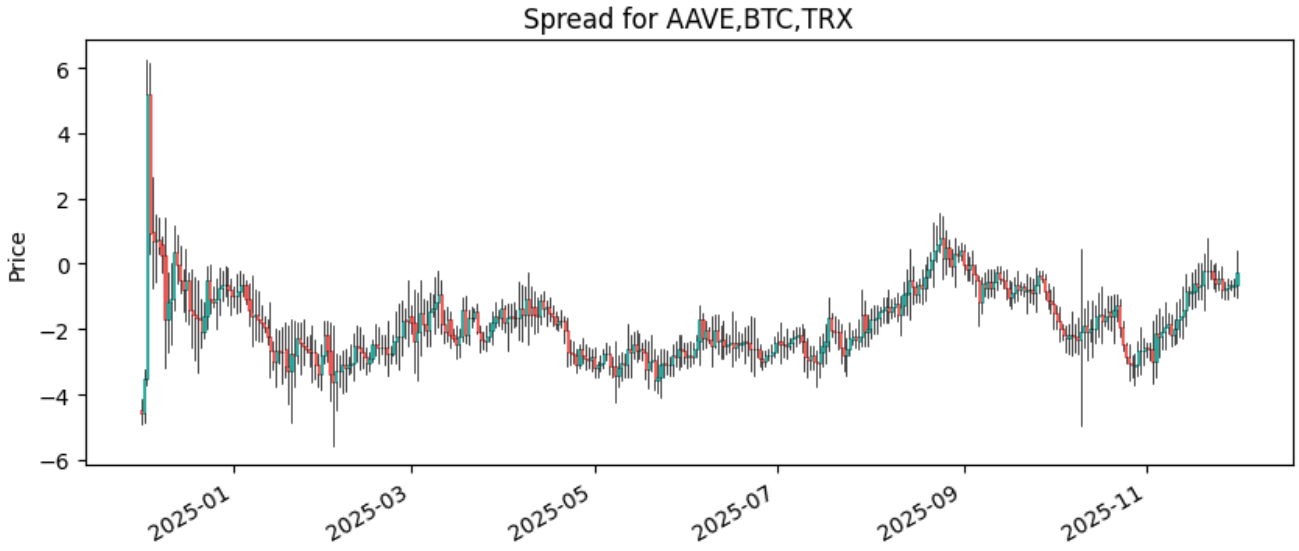
> **نکته مهم:** برای اینکه بتوانیم روند قیمت کوین‌های مختلف (مثلا بیت‌کوین ۱۰۰ هزار دلاری و کاردانو ۱ دلاری) را در یک نمودار مقایسه کنیم، داده‌ها را **نرمالایز** کردیم. یعنی تمام قیمت‌ها را بر قیمت روز اول تقسیم کردیم تا همه از نقطه `1.0` شروع شوند و فقط درصد تغییرات قابل مشاهده باشد. - **نمودار دوم (Spread):** همان کندل‌های ساختگی (Synthetic OHLC) که در مرحله قبل ساختیم را نمایش می‌دهیم تا ببینیم رفتار اسپرد نهایی به چه صورت است.

با اجرای این بخش، خروجی نهایی شامل ۵ نمودار دوتایی (اجزا و اسپرد) به همراه مقادیر دقیق `Half-Life` و `ADF P-value` خواهد بود.

چند نمونه از خروجی: (برای کوتاه کردن داک کل عکس هارو قرار نمیدم)

Normalized constituents for AAVE,BCH,TRX
AAVE
1.5
1.0
0.5
BCH
1.2
1.0
0.8
0.6
TRX
2.00
1.75
1.50
1.25
1.00
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11


Spread for AAVE,BCH,TRX
Price
12
10
8
6
4
2
0
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11



بخش چهارم

در این بخش من دیگر به توضیح استراتژی نمیپردازم چون خیلی کامل در فایل تمرین توضیح داده شده.

فقط یک مورد در داک خوب مشخص نشده بود. بعد از اینکه در سطح اول وارد شدیم و پولمون نصف شد، در سطح دوم کل پول باقی مانده یا دوباره نصف آن وارد میشیم. من فرض کردم در سطح دوم کل

پول باقی مانده درنظر گرفتم.

ابتدا باید اندیکاتورهای خود را محاسبه و مشخص کنیم تا ازش بتونیم استفاده کنیم:

```
def calculate_indicators(spread_ohlc, hl):
    if hl < 30:
        lookback = int(np.ceil(30 / hl) * hl) #The smallest multiple of
`hl` that is ≥ 30
    else:
        lookback = int(2 * hl)

    hl_diff = spread_ohlc['High'] - spread_ohlc['Low']
    ema = hl_diff.ewm(span=lookback, adjust=False).mean()
    std = spread_ohlc['Close'].rolling(window=lookback).std()

    return ema, std, lookback
```

حال باید قسمت سخت این بخش رو پیاده سازی کنیم، ران کردن استراتژی سوال روی دیتاهامون.
در کد زیر دقیقا استراتژی که گفته شده رو پیاده کردم و با کامنت بخش هارو مشخص کردم پس برای کوتاهی داک دیگر توضیحشون نمیدم.
فقط یک مورد اضافه هم چک کنیم که وقتی پولمون تمدم شد، دیگه ادامه ندیم.

```
def run_simulation_loop(spread_ohlc, ema, std, start_idx,
initial_capital):
    capital = initial_capital
    position = 0
    position_value = 0
    entry_price = 0
    trades = []
    equity_curve = []
    is_bankrupt = False

    for i in range(start_idx, len(spread_ohlc)):
        current_date = spread_ohlc.index[i]
        high = spread_ohlc['High'].iloc[i]
        low = spread_ohlc['Low'].iloc[i]
        close = spread_ohlc['Close'].iloc[i]
        current_ema = ema.iloc[i]
        current_std = std.iloc[i]


Read the full in notebook...
```

حال برای تحلیل و مقایسه استراتژی یک فانکشن برای محاسبه *sharpe* و *drawdown* و یک فانکشن برای پلات کردن اضافه کردم.

```
def calculate_metrics(trades, equity_curve, initial_capital,
final_capital, is_bankrupt, risk_free_rate=0.03):
    total_return = ((final_capital - initial_capital) / initial_capital) *
100
    equity_df = pd.DataFrame(equity_curve)


Read the full in notebook...

def plot_results(spread_ohlc, ema, std, trades, metrics, names,
is_bankrupt, initial_capital):
    equity_df = metrics['equity_df']


Read the full in notebook...
```

یک نکته که باید در نظر گرفت این است که چون محاسبات EMA بر اساس اختلاف High و Low (نوسان قیمت) انجام شده ، مقادیر عددی کوچکی هستند، ولی قیمت Close مقادیر بسیار بزرگتری دارد، این دو سری داده روی یک محور مشترک قابل مقایسه نبودند و نمودار به هم ریخته دیده می‌شد.

برای حل این مشکل، ما در تابع `plot_results` از قابلیت Dual Axis (محور دوم Y) استفاده کردیم. به این صورت که قیمت (`Close`) روی محور سمت چپ و اندیکاتورها (`EMA` و باندها) روی محور سمت راست ترسیم می‌شوند تا هر دو با مقیاس صحیح و به طور همزمان قابل مشاهده باشند.

برای حل این مشکل، ما در تابع plot_results از قابلیت Dual Axis (محور دوم Y) استفاده کردیم. به این صورت که قیمت (Close) روی محور سمت چپ و اندیکاتورها (EMA و باندها) روی محور سمت راست ترسیم می‌شوند تا هر دو با مقیاس صحیح و به طور همزمان قابل مشاهده باشند.

بخش پنجم

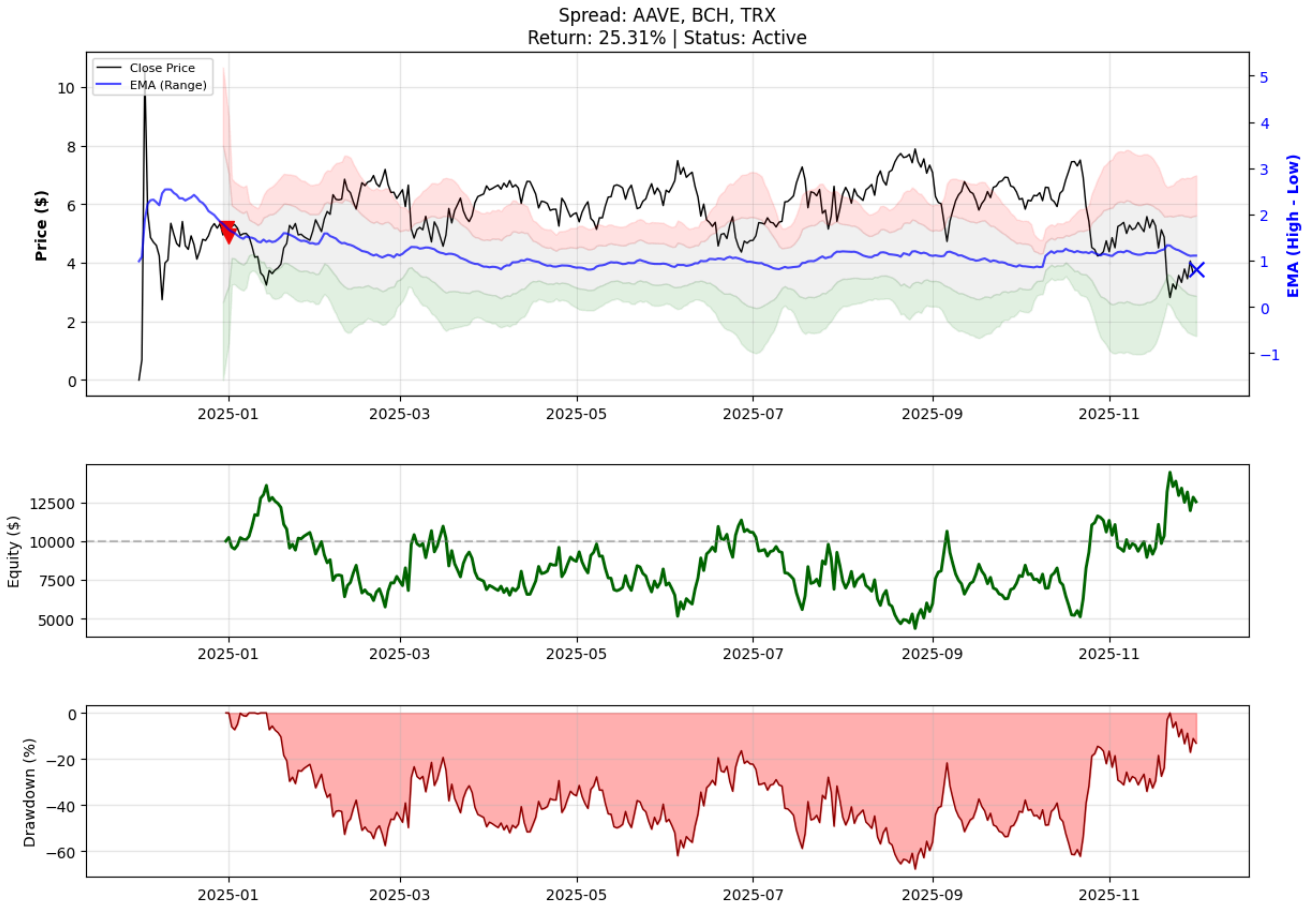
در این بخش استراتژی بخش قبلی را بک تست می‌کنیم

```
def backtest_mean_strategy(top5_spreads, initial_capital=10000):
    all_results = []

    for idx, item in enumerate(top5_spreads):
        names = item['names']
        hl = item['hl']
        spread_ohlc = item['spread_ohlc'].copy()


Read the full in notebook...
```

حال یک سری از نتایجی که گرفتم را نمایش می‌دهم:

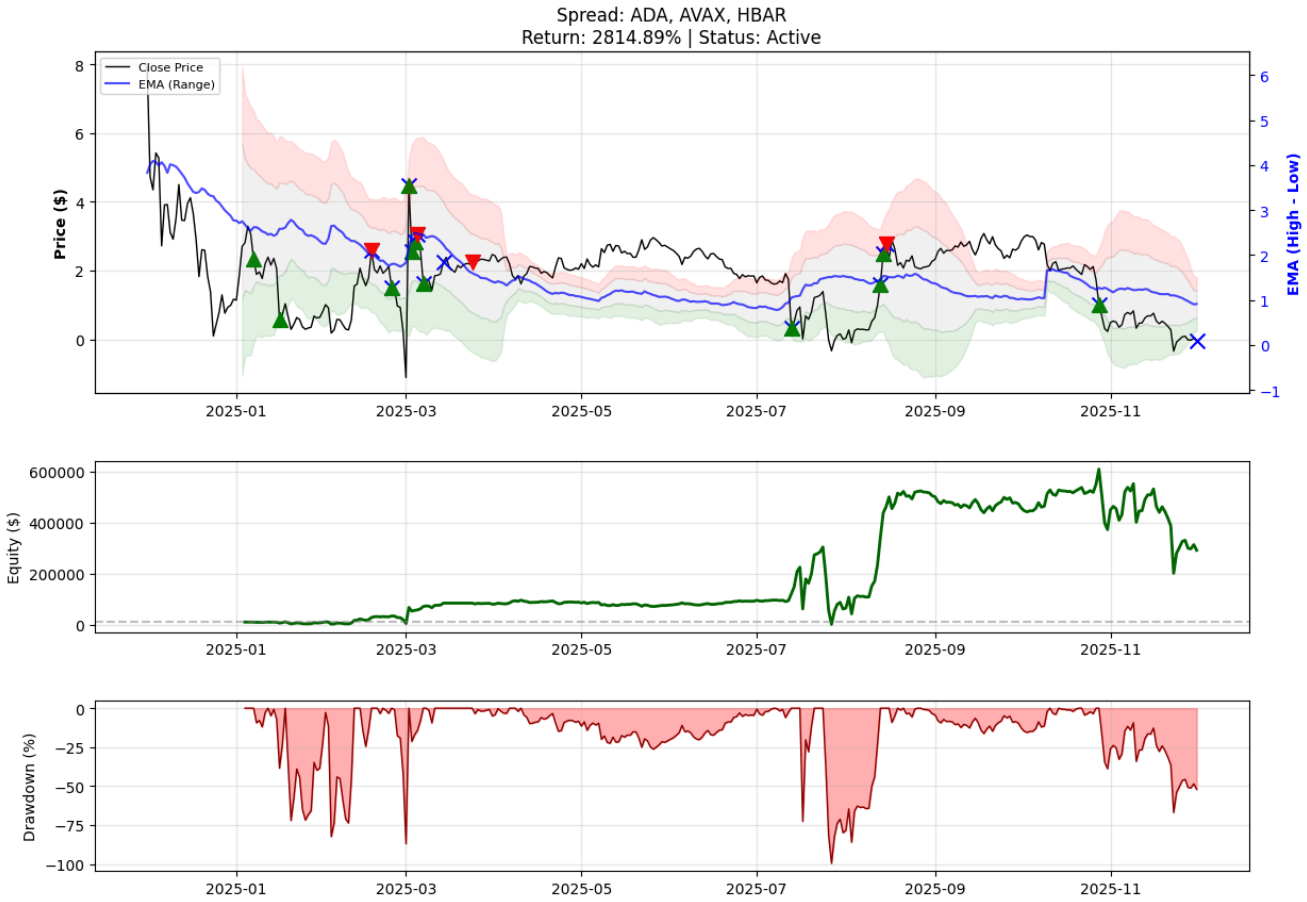


که مشخص است تنها سه ترید انجام داده و نشان گر مناسبی نمی‌باشد.

```
==========================================================================
======
Spread 1: AAVE, BCH, TRX
Half-life: 2.51 days

============================================================
Performance Metrics:
  Initial Capital: $10000.00
  Final Capital: $12530.52
  Total Return: 25.31%
  Sharpe Ratio: 0.9603
  Max Drawdown: -67.79%
  Number of Trades: 3
============================================================

Trade Log (first 10 trades):
    1. 2024-12-31: SHORT_1    @ 5.172228
    2. 2025-01-01: SHORT_2    @ 4.928318
    3. 2025-12-01: FINAL_EXIT @ 3.770090
```



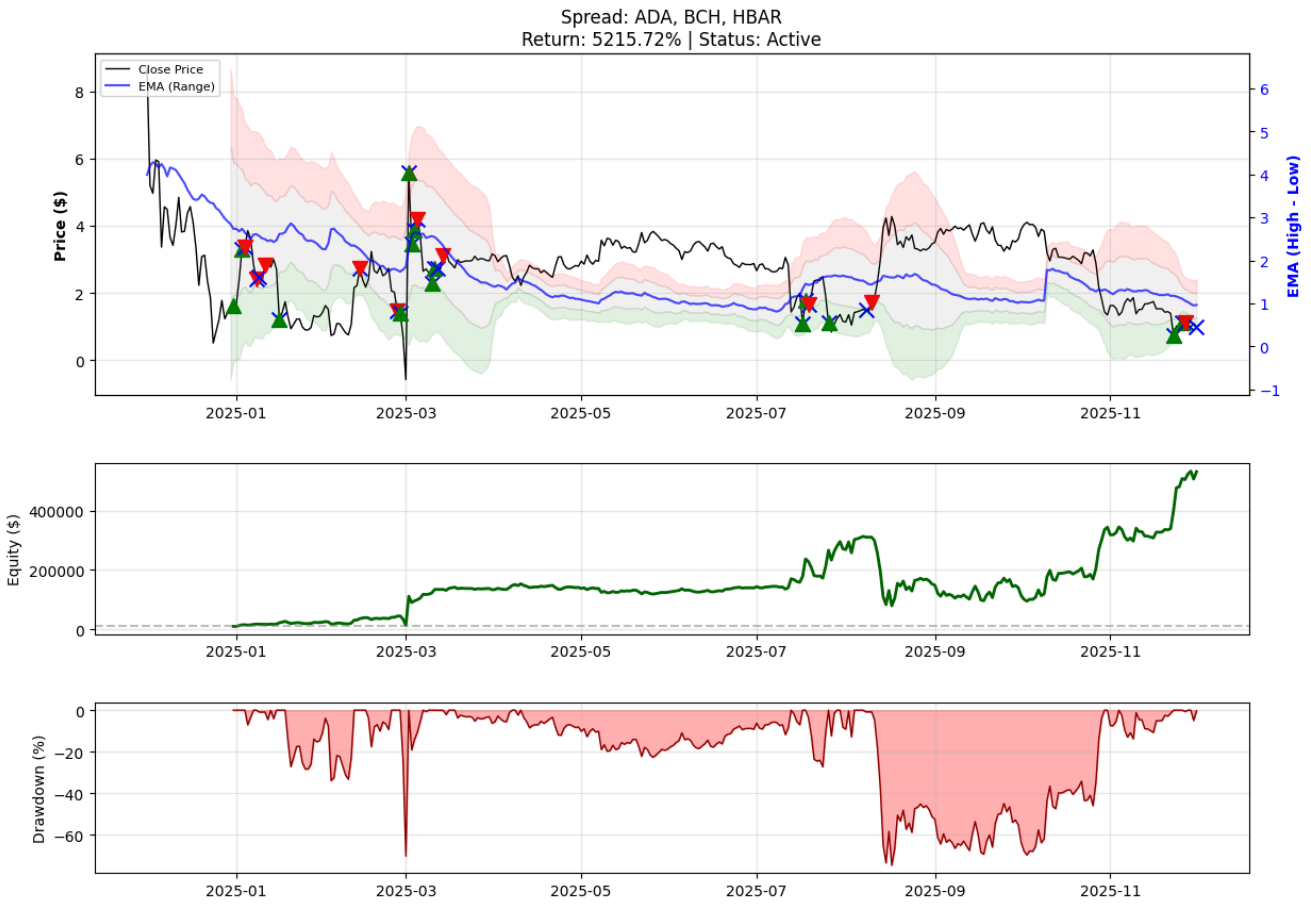

```
==============================================================================
======
Spread 5: ADA, AVAX, HBAR
Half-life: 4.25 days

============================================================
Performance Metrics:
  Initial Capital: $10000.00
  Final Capital: $291489.37
  Total Return: 2814.89%
  Sharpe Ratio: 1.5208
  Max Drawdown: -99.60%
  Number of Trades: 29
============================================================

Trade Log (first 10 trades):
    1. 2025-01-07: LONG_1     @ 2.328084
    2. 2025-01-16: LONG_2     @ 0.574270
    3. 2025-02-17: EXIT       @ 2.581613
    4. 2025-02-17: SHORT_1    @ 2.581613
    5. 2025-02-24: EXIT       @ 1.505361
    6. 2025-02-24: LONG_1     @ 1.505361
    7. 2025-03-02: EXIT       @ 4.491170
    8. 2025-03-02: LONG_1     @ 4.491170
    9. 2025-03-03: EXIT       @ 2.567558
```

```
    10. 2025-03-03: LONG_1     @ 2.567558
```

این یکی بیش از حد توقع خوب عمل کرد:




```
==========================================================================
======
Spread 4: ADA, BCH, HBAR
Half-life: 3.87 days

============================================================
Performance Metrics:
  Initial Capital: $10000.00
  Final Capital: $531571.96
  Total Return: 5215.72%
  Sharpe Ratio: 1.5620
  Max Drawdown: -74.65%
  Number of Trades: 48
============================================================

Trade Log (first 10 trades):
    1. 2024-12-31: LONG_1     @ 1.635892
    2. 2025-01-03: EXIT       @ 3.320567
    3. 2025-01-03: LONG_1     @ 3.320567
    4. 2025-01-04: EXIT       @ 3.381104
    5. 2025-01-04: SHORT_1    @ 3.381104
```

```
    6. 2025-01-08: EXIT       @ 2.425151
    7. 2025-01-08: SHORT_1    @ 2.425151
    8. 2025-01-09: EXIT       @ 2.470583
    9. 2025-01-11: SHORT_1    @ 2.818478
    10. 2025-01-16: EXIT       @ 1.220973
```

و خلاصه متریک ها برای پنج سری تست شده به صورت زیر درمی‌آید:

```
================================================================================
==========================
SUMMARY OF ALL 5 SPREADS
================================================================================
==========================
Spread                                      Return %     Sharpe   Max DD %
Trades
--------------------------------------------------------------------------------
--------------------------
AAVE, BCH...                                  25.31     0.9603     -67.79
3
AAVE, ETH...                                  -0.26     0.2466     -44.23
3
AAVE, BTC...                                -100.00     0.5267    -100.00
463
ADA, BCH...                                 5215.72     1.5620     -74.65
48
ADA, AVAX...                                2814.89     1.5208     -99.60
29
================================================================================
==========================
AVERAGE                                     1591.13     0.9633     -77.25
================================================================================
==========================
```

بخش ششم

مشکل اصلی که در این بخش خوردم، انتخاب مناسب بین سری ها بود.
یک مورد که برای این اضافه کردم که استاد هم سر کلاس گفته بودند، میزان نوسانات(بالا و پایین رفتن) هست

```
def calculate_zero_crossings(series):
    centered = series - series.mean()
    return len(np.where(np.diff(np.sign(centered)))[0])
```

فانکشن انتخاب:

```
def select_best_combo_for_window(selection_start, selection_end,
data_dir=DATA_SAVE_DIR, alpha=0.05, max_size=5):

    found = find_linear_stationary_combinations(
        data_dir=data_dir,
        alpha=alpha,
        target=100,
        max_size=max_size,
        start_day=selection_start,
        end_day=selection_end,
        shuffle=True
    )



Read the full in notebook...
```

منطق انتخاب

۱. تعداد Zero Crossings

```
z_crossings = calculate_zero_crossings(spread_series)
```

این متریک تعداد دفعاتی را می‌شمارد که اسپرد از میانگین خود عبور می‌کند (بالا و پایین می‌رود).
در امتیازدهی، این مهمترین فاکتور است:

```
score += z_crossings * 1.5
```

۲. فیلتر کردن اسپردهای کم‌نوسان

```
if z_crossings < (days_in_sample / 60):
    continue
```

اگر در طول ۶ ماه، اسپرد کمتر از ۳ بار از میانگین عبور کرده باشد، آن را رد می‌کنیم.

۳. چک کردن واریانس

```
spread_std = spread_series.std()
if spread_std < 1e-4:
    continue
```

اسپردهایی که واریانس بسیار کمی دارند را حذف می‌کنیم :

- ممکن است Numerically Unstable باشند

- احتمالا Arbitrage یا ترکیب‌های تقریبا یکسان هستند
- در عمل قابل معامله نیستند

۴. پنالتی برای نوسان افراطی

```
cv = abs(spread_std / spread_mean)
if cv > 3.0:
    score -= 20
```

- اسپرد بسیار ناپایدار است
- ممکن است Half-Life واقعی آن خیلی متفاوت از محاسبه‌شده باشد
- ریسک Regime Change بالاست

به همین دلیل امتیاز کم می‌گیرد.

و در آخر باید *p-value* باید کمتر از ۰.۰۵ باشد وگرنه از امتیازش کم میکنیم.

فانکشن **build_spread_ohlc** همانند **select_top5_and_plot** برای ما OHLC را درست میکند فقط چون پلات کردن نیاز نداشتیم آن را در یک فانکشن جدا قرار دادم.

حال سراغ فانکشن اصلی و پیاده سازی خود منطق جدید می‌رویم:

```
def backtest_single_spread(item, trade_start, trade_end,
initial_capital=10000, plot=False):
    if initial_capital ≤ 0:
        print(f"Cannot backtest with zero or negative capital:
${initial_capital:.2f}")
        return None

    names = item['names']
    vec = item['vec']
    hl = item.get('hl', np.nan)

Read the full in notebook...
```

توضیح فانکشن backtest_single_spread

این فانکشن یک اسپرد را در بازه زمانی مشخص (`trade_start` تا `trade_end`) بک‌تست می‌کند و نتایج کامل را برمی‌گرداند.

تفاوت‌های کلیدی با بخش ۴:

1. **محاسبه EMA:** طبق توضیح دکتر انتظاری در کلاس، EMA را مستقیما روی قیمت `Close` محاسبه می‌کنیم (نه اختلاف High-Low)

```
    ()ema = spread_ohlc['Close'].ewm(span=lookback, adjust=False).mean
```

2. **Trade Window با Historical Data:**
   - ۶ ماه قبل از `trade_start` داده لود می‌کنیم برای به دست آوردن اندیکاتورها
   - اما فقط در بازه `(trade_start, trade_end]` اجازه ورود/خروج معامله داریم
3. **Position Management:**
   - `position_direction` : +1 برای long، -1 برای short، 0 برای flat
   - `position_value` : مقدار نوتیونال سرمایه‌ای که در پوزیشن است
   - محاسبه PnL بر اساس تغییرات اسپرد نسبت به قیمت ورود
4. **Level 2 Entries:**
   - وقتی در Level 2 (±2σ) سیگنال بگیریم، **تمام** سرمایه باقی‌مانده را وارد می‌کنیم
   - قیمت ورود را به صورت میانگین وزنی (weighted average) به‌روزرسانی می‌کنیم
5. **Capital Checks:**
   - بررسی liquidation: اگر ارزش portfolio به صفر یا منفی برسد
   - بررسی capital depletion: اگر پول نقد تمام شود
   - در هر دو حالت، معامله متوقف می‌شود
6. **Output:**
   - `position_open_end` : آیا در پایان بازه پوزیشن باز داریم؟
   - متریک‌های کامل: Total Return، Max Drawdown، Sharpe
   - لیست کامل معاملات و equity curve

خب حالا باید سراغ ران کردن و محاسبه متریک های لازم برای آن بشویم:

```
def handle_extended_position(chosen, trade_start, trade_end,
month_start_capital, res):
    print("position remained open at month end. Waiting until it closes
before reselection")

    search_start = trade_end
    search_limit = trade_end + rd.relativedelta(months=6)
    closed = False

    final_capital = float(res.get('final_capital', month_start_capital))
    next_date = trade_end

Read the full in notebook...
```

این فانکشن یکی از مهم‌ترین بخش‌ها است و مسئولیت مدیریت پوزیشن‌هایی است که در پایان ماه همچنان باز هستند.

حال برای پیدا کردم زمان بسته شدن پوزیشن:

1. **Search Window:** تا ۶ ماه آینده را جستجو می‌کنیم تا ببینیم پوزیشن کی بسته میشه

```
search_limit = trade_end + rd.relativedelta(months=6)
```

۲. **Incremental Backtesting:**

- روز به روز بازه بک‌تست را extend می‌کنیم
- در هر تکرار، بک‌تست را از `trade_start` تا `search_start + 1 day` اجرا می‌کنیم
- این کار را تا زمانی ادامه می‌دهیم که `position_open_end == False` شود

۳. **Finding Exit Date:**

- وقتی پوزیشن بسته شد، باید دقیقا تاریخ خروج را پیدا کنیم
- لیست معاملات را از آخر جستجو می‌کنیم تا اولین `EXIT` را پیدا کنیم
- تاریخ شروع ماه بعدی = روز بعد از Exit

۴. **Timeout Handling:**

- اگر بعد از ۶ ماه پوزیشن هنوز باز بود، جستجو را متوقف می‌کنیم
- سرمایه نهایی را از آخرین نتیجه می‌گیریم
- به ماه بعدی می‌رویم (بدون انتخاب مجدد)

> این بخش رو گذاشتم که یه پوزیشن باز تا اخر نره جلو و کل استراتژی رو خراب نکنه.

فانکشن‌های خروجی و گزارشگیری

`print_metrics` : تمام نقاط equity و Sharpe Ratio و Max Drawdown کلی را از همه ماه‌ها جمع‌آوری می‌کند و سیستم را محاسبه و چاپ می‌کند.

`print_period_breakdown` : چند، هر ماه کدام اسپرد انتخاب شد، جدول ماه‌به‌ماه نتایج را نمایش می‌دهد - هر کدام چند بار انتخاب شدند، وزن‌هاشان چیست، و معامله انجام شد، و بازده و سرمایه نهایی چقدر بود.

`print_unique_combinations` : لیست تمام اسپردهای یونیکی که در طول دوره انتخاب شدند را نمایش می‌دهد - هر کدام Sharpe Ratio مخصوص به خودشان چقدر است.

که سه مورد بالا در `get_report` صدا زده می‌شوند.

فانکشن run_new_strategy

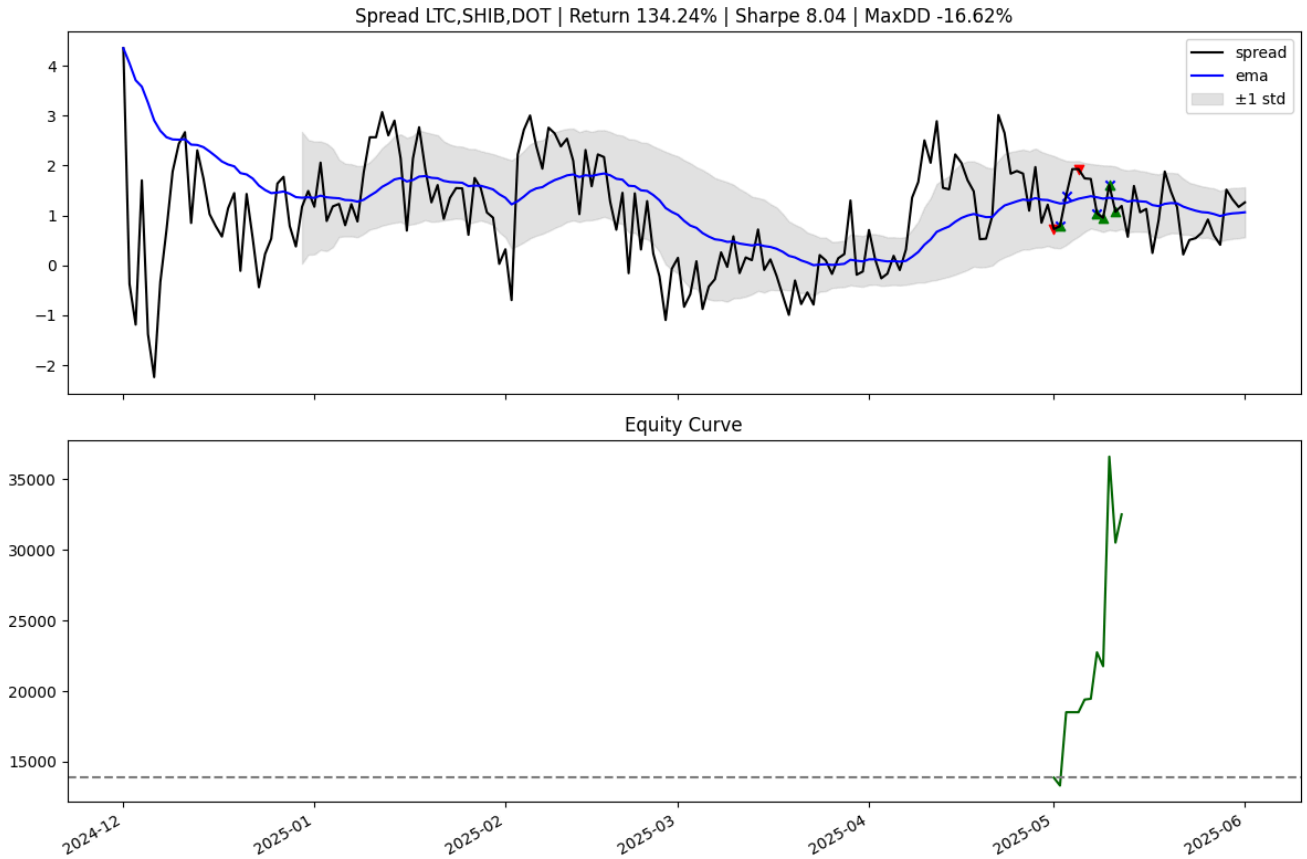
این فانکشم کل موارد این بخش رو کنترل و با یکدیگر سازگار می‌کند.

1. از ۶ ماه گذشته بهترین اسپرد را انتخاب می‌کند (`select_best_combo_for_window`)
2. آن اسپرد را برای یک ماه آینده معامله می‌کند (`backtest_single_spread`)
3. اگر پوزیشن باز ماند، منتظر بسته شدن آن می‌ماند (`handle_extended_position`)
4. سرمایه را به‌روز می‌کند و به ماه بعدی می‌رود

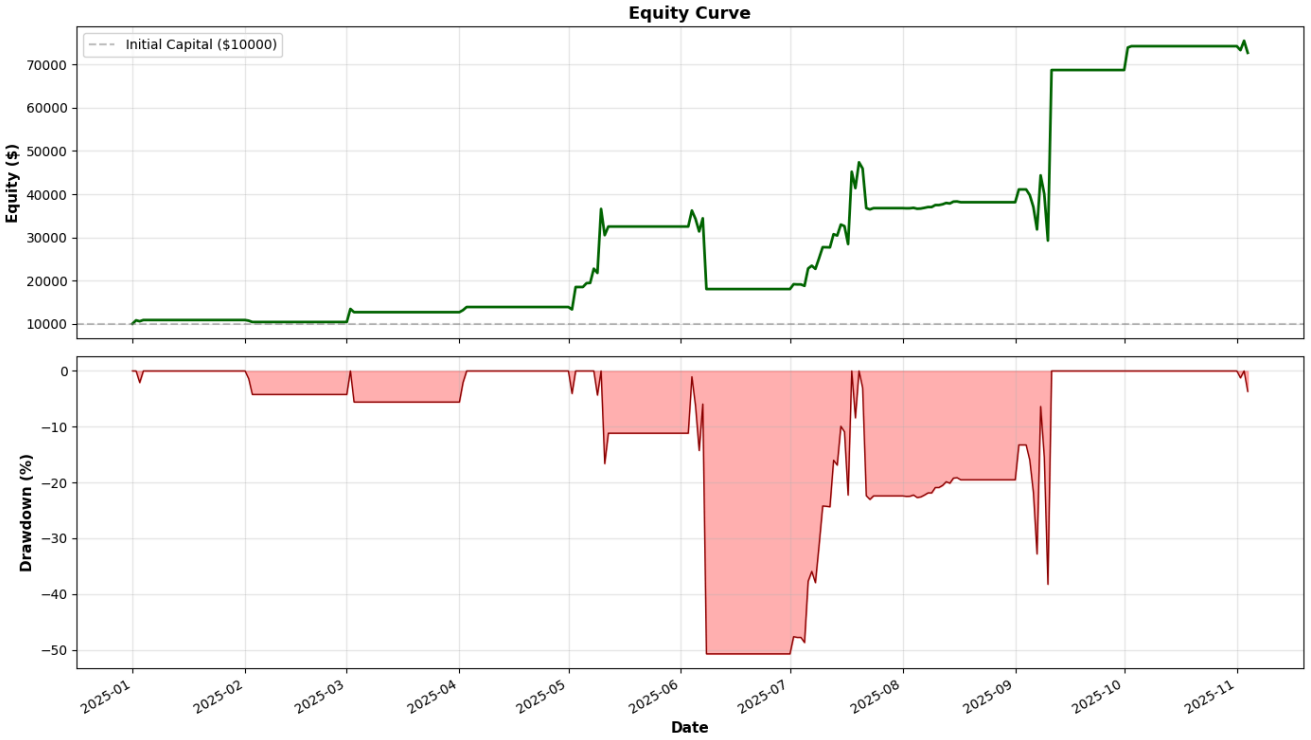
در پایان، لیست کامل نتایج تمام ماه‌ها را برمی‌گرداند که شامل اسپردهای انتخابی، معاملات انجام شده، و سرمایه نهایی هر دوره است.

نتایج اجرا:

نتایج هرماه در نوتبوک مشخص است من تنها یک مورد را برای نمایش نشان می‌دهم:



درآخر هم سرمایه و متریک ها به صورت زیر می‌شوند:




```
Period: 2025-01-01 to 2025-12-01
Initial Capital: $10000.00
Final Capital: $72719.77
Total Return: 627.20%
Number of Selection Periods: 11
Total Trades: 83
Overall Sharpe Ratio: 3.4942
```

```
Overall Max Drawdown: -50.71%
================================================================================
PERIOD-BY-PERIOD BREAKDOWN
================================================================================
Period                  Combination                                Trades
Return %    Final Cap
--------------------------------------------------------------------------------
2025-01-01 to 2025-02-01 ENA, SHIB, NEAR                                4
8.65 $   10865.11
2025-02-01 to 2025-03-01 XRP, LINK, BTC                                 2
-4.22 $   10406.38
2025-03-01 to 2025-04-01 ETC, UNI, SHIB                                 2
21.88 $   12682.85
2025-04-01 to 2025-05-01 WLD, ICP, SHIB                                 2
9.42 $   13877.26
2025-05-01 to 2025-06-01 LTC, SHIB, DOT                                11
134.24 $   32506.43
2025-06-01 to 2025-07-01 HBAR, AVAX, SHIB                               4
-44.52 $   18034.25
2025-07-01 to 2025-08-01 PEPE, AVAX, SHIB                              34
103.86 $   36765.08
2025-08-01 to 2025-09-01 BTC, AAVE, LEO                                16
3.72 $   38133.09
2025-09-01 to 2025-10-01 HBAR, ETC, LTC                                 4
80.25 $   68736.26
2025-10-01 to 2025-11-01 KCS, SHIB, DOT                                 2
8.03 $   74255.26
2025-11-01 to 2025-12-01 XRP, SUI, SHIB                                 2
-2.07 $   72719.77
================================================================================

1. ENA, NEAR, SHIB
   Selected 1 time(s): 2025-01
   Weights: 3.5385, 2178981.1428, -8.4214
   Half-life: nan, Hurst: -0.2054
   Sharpe Ratio: 10.8982

2. BTC, LINK, XRP
   Selected 1 time(s): 2025-02
   Weights: 5.1060, 0.9123, -0.0009
   Half-life: 0.34, Hurst: 0.0030
   Sharpe Ratio: -39.6064

3. ETC, SHIB, UNI
   Selected 1 time(s): 2025-03
   Weights: 0.3932, 0.8672, -939913.8211
```

```
   Half-life: 1.62, Hurst: 0.2655
   Sharpe Ratio: 9.1503

4. ICP, SHIB, WLD
   Selected 1 time(s): 2025-04
   Weights: 3.9205, 2.3371, -1746047.7687
   Half-life: 0.67, Hurst: 0.1425
   Sharpe Ratio: 71.0535

5. DOT, LTC, SHIB
   Selected 1 time(s): 2025-05
   Weights: 0.0054, 1554600.6092, -4.7798
   Half-life: 1.47, Hurst: 0.1536
   Sharpe Ratio: 8.0360

6. AVAX, HBAR, SHIB
   Selected 1 time(s): 2025-06
   Weights: 6.3640, -0.8038, 1485675.2075
   Half-life: 1.14, Hurst: 0.0712
   Sharpe Ratio: -5.5579

7. AVAX, PEPE, SHIB
   Selected 1 time(s): 2025-07
   Weights: 71464.8411, -1.0033, 1894086.9014
   Half-life: 1.81, Hurst: 0.0988
   Sharpe Ratio: 5.1568

8. AAVE, BTC, LEO
   Selected 1 time(s): 2025-08
   Weights: 0.0003, -0.0539, 3.9433
   Half-life: 2.21, Hurst: 0.0576
   Sharpe Ratio: 8.1028

9. ETC, HBAR, LTC
   Selected 1 time(s): 2025-09
   Weights: 19.2064, -1.4514, 0.2158
   Half-life: 2.14, Hurst: 0.0556
   Sharpe Ratio: 4.9828

10. DOT, KCS, SHIB
   Selected 1 time(s): 2025-10
   Weights: 0.3423, 2099647.6399, -5.6936
   Half-life: 1.77, Hurst: 0.1410
   Sharpe Ratio: 15.0696

11. SHIB, SUI, XRP
   Selected 1 time(s): 2025-11
   Weights: 2.8350, -6.7379, 1853315.9239
   Half-life: 1.51, Hurst: 0.0547
   Sharpe Ratio: -3.7796
```

```
==========================================================================
======
```